



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

JavaScript

Angelo Sotgiu

angelo.sotgiu@unica.it

In questa lezione

- Concetti e costrutti di base JavaScript
- AJAX
- Manipolazione dinamica HTML



Cos'è JavaScript

Definizione

- Linguaggio di scripting interpretato, dinamico, debolmente tipizzato
- Standardizzato come ECMAScript ([ES6](#) e versioni successive)

Storia

- Creato nel 1995 da Brendan Eich (Netscape) in 10 giorni
- Oggi è uno dei linguaggi più usati al mondo

Come funziona JavaScript

Esecuzione

- Eseguito da un motore JS (es. **V8** in Chrome / Node.js)
- Single-threaded con event loop: un solo thread esegue il codice e una coda smista i task asincroni (timer, risposte di rete) man mano che arrivano

Caratteristiche

- Operazioni I/O non bloccanti (asincrone)
- Interpretato lato browser, senza compilazione esplicita

Dove e quando si usa

Lato client (browser)

- Pagine interattive, validazione form, animazioni
- Pattern Code on Demand
- Single Page Applications (SPA)

Lato server

- Node.js per API REST, microservizi, tooling

Altri ambiti

- Mobile (React Native), desktop (Electron), IoT



Sintassi: variabili e tipi

Dichiarazione

- `let nome = 'Alice'` — visibile solo dentro le `{ }` in cui è dichiarata, modificabile
- `const PI = 3.14` — costante: non si può riassegnare un nuovo valore alla variabile
- `var x = 1` — legacy: visibile in tutta la funzione, da evitare

Tipi primitivi

- `string, number, boolean, null, undefined`
- Tipizzazione dinamica: `typeof x` restituisce il tipo del valore corrente

Sintassi: operatori e control flow

Operatori

- Aritmetici: `+` `-` `*` `/` `%` `**` Logici: `&&` `||` `!`

Confronto: `==` vs `===`

- `==` converte gli operandi a un tipo comune prima del confronto (es. stringa $\rightarrow$ numero, booleano $\rightarrow$ numero)
- `'1' == 1  $\rightarrow$  true` `0 == false  $\rightarrow$  true`
- `===` confronta valore e tipo (raccomandato)
- `'1' === 1  $\rightarrow$  false`

Control flow

- `if / else if / else`, `switch`, cicli `for` / `while` / `for...of`

Sintassi: funzioni

Dichiarazione classica

```
– function somma(a, b) {  
    return a + b;  
}
```

Funzione anonima / espressione

```
– const somma = function(a, b) { return a + b; }
```

Arrow function

- `const somma = (a, b) => a + b;`
- Le funzioni sono oggetti di prima classe (passabili come argomenti)

Sintassi: oggetti e array

Oggetti

- Coppie chiave-valore: `const libro = { titolo: 'IT', autore: 'King' }`
- Accesso: `libro.titolo` oppure `libro['titolo']`

Array

- Lista ordinata: `const nums = [1, 2, 3];`
- Iterazione con `.forEach()`: esegue una funzione per ogni elemento dell'array
`nums.forEach(n => console.log(n));`
n è il parametro della funzione (nome arbitrario): a ogni giro vale 1, poi 2, poi 3
- Altri metodi: `.map()`, `.filter()`

Asincronia: Promise e async/await

Asincronia

- Le operazioni I/O (**fetch**, timer) non bloccano il thread

Promise

- Oggetto che rappresenta un valore futuro (pending / fulfilled / rejected)
- È il valore restituito da una funzione asincrona: la funzione termina subito e il Promise si risolve quando il risultato è pronto

async / await

- Sintassi che permette di scrivere codice asincrono come se fosse sincrono
- **const data = await fetch(url);**



Embedding in HTML

Modi per includere JS

- Inline: `<script> ... </script>` direttamente in HTML
- Esterno: `<script src="app.js"></script>` (raccomandato)

Posizione e caricamento

- In `<head>` senza attributi: blocca il parsing dell'HTML
- `defer` e `async` sono attributi del tag `<script src="...">` nell'`<head>`
 - `defer`: scarica in parallelo, esegue dopo il parsing (mantiene l'ordine) — adatto a script che usano il DOM
 - `async`: scarica in parallelo, esegue appena pronto (ordine non garantito) — adatto a script indipendenti come analytics
- In fondo al `<body>`: il DOM è già disponibile

AJAX e fetch (recap)

AJAX (recap)

- Asynchronous JavaScript And XML: richieste HTTP dal browser
- Aggiorna parti della pagina senza ricaricarla. Oggi i dati viaggiano in JSON

fetch — GET

- `const r = await fetch('/books');`
- `if (!r.ok) throw new Error(r.status);`
- `const data = await r.json();`

fetch — POST con JSON

- `await fetch('/books', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ titolo: 'IT', autore: 'King' }) });`



DOM: selezionare elementi

Document Object Model

- Rappresentazione ad albero della pagina HTML: ogni tag annidato diventa figlio del tag che lo contiene (es. `html` → `body` → `div` → `p`);
- `document` è la variabile globale che rappresenta la radice dell'albero

Funzioni di selezione

- Se un elemento HTML ha l'`id` (es. `<div id="menu">`), può essere selezionato con:
`document.getElementById('menu')`
- Altrimenti, può essere selezionato tramite tag (es. `'div'`), classe (es. `'.menu'`) o per posizione/relazione (es. `'header p'`) con:
`document.querySelector('sel')` — primo elemento che matcha il selettore
`document.querySelectorAll('sel')` — tutti gli elementi che matchano
`document.getElementsByClassName('c')` — in alternativa, per classe



DOM: modificare elementi e gestire eventi

Cambiare il testo

- JS: `document.getElementById('msg').textContent = 'Ciao';`
- Prima: `<p id="msg">...</p>`
- Dopo: `<p id="msg">Ciao</p>`

Cambiare un attributo

- JS: `img.setAttribute('src', 'foto.jpg');`
- Prima: `<img src="">`
- Dopo: ``

Cambiare una classe

- JS: `btn.classList.add('attivo');`
- Prima: `<button>OK</button>`
- Dopo: `<button class="attivo">OK</button>`



DOM: modificare elementi e gestire eventi

Attivare/disattivare una classe

- JS: `box.classList.toggle('attivo');`
- Prima: `<div id="box">...</div>`
- Dopo: `<div id="box" class="attivo">...</div>`

Cambiare uno stile inline

- JS: `p.style.color = 'red';`
- Prima: `<p>Errore</p>`
- Dopo: `<p style="color: red">Errore</p>`

Leggere il valore di un input

- JS: `const nome = campo.value;`
- HTML: `<input id="campo" value="Mario">`
- Risultato: `nome === 'Mario'`



DOM: modificare elementi e gestire eventi

Inserire HTML in un elemento

- JS: `box.innerHTML = '<b>Ciao</b>';`
- Prima: `<div id="box"></div>`
- Dopo:
`<div id="box">`
 `<b>Ciao</b>`
`</div>`

Reagire a un click

- HTML: `<button id="b">Click</button> <p id="out"></p>`
- JS: `b.addEventListener('click', () => {`
 `out.textContent = 'Cliccato!';`
 `});`
al click `<p>` mostra "Cliccato!"

DOM: creare nodi e gestire eventi

Aggiungere un nuovo elemento

- JS: `const li = document.createElement('li');`
 `li.textContent = 'Latte';`
 `document.querySelector('ul').appendChild(li);`
- Prima: `<ul></ul>`
- Dopo:
 `<ul>`
 `<li>Latte</li>`
 `</ul>`

JavaScript: rischi di sicurezza e mitigazioni

XSS — Cross-Site Scripting

- Rischio: input utente inserito come HTML esegue script malevoli
- Mitigazione: preferire `textContent` a `innerHTML`; sanitizzare input; abilitare Content Security Policy (CSP) per limitare le fonti di script

Esecuzione di codice arbitrario

- Rischio: `eval()` esegue stringhe come codice; librerie di terze parti compromesse
- Mitigazione: evitare `eval`; script da fonti fidate; Subresource Integrity (SRI)

Dati sensibili nel client

- Rischio: chiavi API, password o logica di sicurezza nel JS sono visibili a tutti
- Mitigazione: segreti e validazione sul server; il client non è mai affidabile

CSRF e clickjacking

- Rischio: azioni a insaputa dell'utente; pagina caricata in iframe nascosto
- Mitigazione: token CSRF, cookie `SameSite`, header `X-Frame-Options`



Caricare la lista di libri

Cosa fa

- Chiama l'API `GET /books` e recupera l'elenco
- Genera dinamicamente le righe della tabella nel DOM

Punti chiave

- `fetch` + `async/await` per la chiamata HTTP
- Lettura di una checkbox per ordinare i risultati
- Inserimento HTML tramite `innerHTML`

Funzione per caricare la lista di libri

```
<script>
  async function LoadBooks() {
    const sortChecked = document.getElementById('sort-toggle').checked;
    const response = await fetch('/books/?sort=' + sortChecked);
    const books = await response.json();
    const tbody = document.getElementById('books-tbody');
    tbody.innerHTML = '';
    books.forEach(function(book) {
      const row = document.createElement('tr');
      row.innerHTML =
        '<td>' + book.id + '</td>' +
        '<td>' + book.title + '</td>' +
        '<td>' + book.author + '</td>' +
        '<td>' + book.review + '/5</td>' +
        '<td><a class="link-detail" href="/ui/books/' + book.id + '">Dettagli</a></td>';
      tbody.appendChild(row);
    });
  }
  document.addEventListener('DOMContentLoaded', LoadBooks);
  document.getElementById('sort-toggle').addEventListener('change', LoadBooks);
</script>
```

Caricare il dettaglio di un libro

Cosa fa

- Chiama `GET /books/<id>` per ottenere un singolo libro
- Popola i campi della pagina con i dati ricevuti

Punti chiave

- L'ID del libro viene passato lato server tramite Jinja
- Selezione degli elementi con `getElementById`
- Aggiornamento testo con `textContent`

Funzione per caricare il dettaglio di un libro

```
<script>
const BOOK_ID = {{ book.id }};
async function LoadBookDetails() {
  const response = await fetch('/books/' + BOOK_ID);
  const book = await response.json();
  const card = document.getElementById('book-card');
  card.innerHTML =
    '<h2>' + book.title + '</h2>' +
    '<div class="detail-attrs">' +
      '<span class="label">ID:</span><span>' + book.id + '</span>' +
      '<span class="label">Autore:</span><span>' + book.author + '</span>' +
      '<span class="label">Recensione:</span><span>' + book.review + '/5</span>' +
    '</div>';
  card.appendChild(btn);
}
async function LoadBookUsers() {
  const response = await fetch('/books/' + BOOK_ID + '/users');
  const users = await response.json();
  const tbody = document.getElementById('users-tbody');
  tbody.innerHTML = '';
  users.forEach(function(user) {
    const row = document.createElement('tr');
    row.innerHTML =
      '<td>' + user.id + '</td>' +
      '<td>' + user.name + '</td>' +
      '<td>' + user.city + '</td>' +
      '<td><a class="link-detail" href="/ui/users/' + user.id + '>Vedi →</a></td>';
    tbody.appendChild(row);
  });
}
document.addEventListener('DOMContentLoaded', function() {
  LoadBookDetails();
  LoadBookUsers();
});
</script>
```

Eliminare un libro

Cosa fa

- Chiama l'API `DELETE /books/<id>` al click sul pulsante
- Chiede conferma all'utente prima di procedere

Punti chiave

- Listener `click` sul bottone di eliminazione
- `window.confirm()` per la conferma
- Redirect alla lista in caso di successo

Funzione per eliminare un libro

```
<script>
...
async function loadBookDetails() {
    ...
    const btn = document.createElement('button');
    btn.textContent = 'Elimina libro';
    btn.className = 'btn btn-danger';
    btn.addEventListener('click', deleteBook);
    card.appendChild(btn);
}
async function deleteBook() {
    if (!confirm('Eliminare questo libro?')) return;
    const response = await fetch('/books/' + BOOK_ID, { method: 'DELETE' });
    if (response.ok) {
        window.location.href = '/ui/books';
    }
}
</script>
```


Caricare la lista di utenti

Cosa fa

- Chiama `GET /users` per recuperare l'elenco degli utenti
- Genera dinamicamente le righe della tabella

Punti chiave

- Stessa logica della lista libri, applicata a un'altra risorsa
- Iterazione sull'array con `forEach` o `map`

Funzione per caricare la lista di utenti

```
<script>
  async function LoadUsers() {
    const response = await fetch('/users');
    const users = await response.json();
    const tbody = document.getElementById('users-tbody');
    tbody.textContent = '';
    users.forEach(function(user) {
      const row = document.createElement('tr');
      ['id', 'name', 'birth_date', 'city'].forEach(function(key) {
        const td = document.createElement('td');
        td.textContent = user[key];
        row.appendChild(td);
      });
      const tdLink = document.createElement('td');
      const link = document.createElement('a');
      link.className = 'link-detail';
      link.href = '/ui/users/' + user.id;
      link.textContent = 'Dettagli';
      tdLink.appendChild(link);
      row.appendChild(tdLink);
      tbody.appendChild(row);
    });
  }
  document.addEventListener('DOMContentLoaded', LoadUsers);
</script>
```

Caricare il dettaglio di un utente

Cosa fa

- Chiama `GET /users/<id>` per i dati del singolo utente
- Compila la scheda dell'utente nella pagina

Punti chiave

- ID utente fornito dal server tramite Jinja
- Stessa struttura della funzione di dettaglio libro

Funzione per caricare il dettaglio di un utente

```
<script>
const USER_ID = {{ user_id }};
async function loadUserDetails() {
  const response = await fetch('/users/' + USER_ID);
  const user = await response.json();
  const card = document.getElementById('user-card');
  card.textContent = '';
  const h2 = document.createElement('h2');
  h2.textContent = user.name;
  card.appendChild(h2);
  const attrs = [
    { label: 'ID:', value: user.id },
    { label: 'Data di nascita:', value: user.birth_date },
    { label: 'Città:', value: user.city }
  ];
  attrs.forEach(function(a) {
    const label = document.createElement('span');
    label.className = 'label';
    label.textContent = a.label;
    const value = document.createElement('span');
    value.textContent = a.value;
    card.appendChild(label); card.appendChild(value);
  });
  document.addEventListener('DOMContentLoaded', loadUserDetails);
}</script>
```

Eliminare un utente

Cosa fa

- Chiama `DELETE /users/<id>` al click sul pulsante
- Richiede conferma prima dell'eliminazione

Punti chiave

- Listener `click` sul bottone di eliminazione
- Stessa struttura della funzione di eliminazione libro

Funzione per eliminare un libro

```
<script>
  ...
  async function LoadUserDetails() {
    ...
    const btn = document.createElement('button');
    btn.textContent = '🗑 Elimina utente';
    btn.className = 'btn btn-danger';
    btn.addEventListener('click', deleteUser);
    card.appendChild(btn);
  }
  async function deleteUser() {
    if (!confirm('Eliminare questo utente?')) return;
    const response = await fetch('/users/' + USER_ID, { method: 'DELETE' });
    if (response.ok) {
      window.location.href = '/ui/users';
    }
  }
}
</script>
```

Aggiungere un libro

Cosa fa

- Legge i dati dal form HTML al submit
- Invia una **POST /books** con il nuovo libro
- Redirect alla lista libri in caso di successo

Punti chiave

- **`event.preventDefault()`** per evitare il submit nativo
- **`FormData`** o lettura diretta dei campi
- Invio del corpo come JSON con header **`Content-Type`**

Funzione per aggiungere un libro

```
<script>
  async function submitBook(event) {
    event.preventDefault();
    const bookData = {
      title: document.getElementById('title').value,
      author: document.getElementById('author').value
    };
    const reviewValue = document.getElementById('review').value;
    if (reviewValue !== '') {
      bookData.review = parseInt(reviewValue);
    }
    const response = await fetch('/books/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(bookData)
    });
    if (response.ok) {
      window.location.href = '/ui/books';
    }
  }

  document.getElementById('add-book-form').addEventListener('submit', submitBook);
</script>
```